

ALGORITHM

Spring 2026, Quiz 10, May 20

11:45-12:05 am

10

姓名：黃翊宏
學號：01357101
系所：CS 2B

1. Consider the longest increasing subsequence problem defined as follows. Given a list of numbers $a_1, \dots, a_n$ an increasing subsequence is a list of indices $i_1, \dots, i_k \in \{1, \dots, n\}$ such that $i_1 < i_2 < \dots < i_k$ and $a_{i_1} \leq a_{i_2} \leq \dots \leq a_{i_k}$. The longest increasing subsequence is the longest list of indices with this property.

- What is the longest increasing subsequence of the list 5, 3, 4, 8, 7, 10?
- Consider the greedy algorithm that chooses the first element of the list, and then repeatedly chooses the next element that is larger. Is this a correct algorithm? Either prove its correctness or provide a counter example.
- Design a dynamic programming algorithm for longest increasing subsequence. Prove its correctness and analyze its running time.

(a) $\langle 3, 4, 8, 10 \rangle$ or $\langle 3, 4, 7, 10 \rangle$, 長度為 4。

(b) 此 Greedy Algorithm 不正確。

依然考慮此序列： $\langle 5, 3, 4, 8, 7, 10 \rangle$

→ 選擇第一個元素 5

→ 選擇下一個比 5 大的元素：8

→ 選擇下一個比 8 大的元素：10

→ 沒有更多的元素可以選，演算法結束，結果為 $\langle 5, 8, 10 \rangle$ ，長度為 3。

真正的 LIS 應為 $\langle 3, 4, 8, 10 \rangle$ or $\langle 3, 4, 7, 10 \rangle$ ，長度為 4。

∴ 存在至少一個比 Greedy 解更佳的 LIS，此 Greedy Algorithm 未能找到最佳解。

∴ 此 Greedy Algorithm 不正確。

(c) 寫在背面

(C)

LIS($a[1 \dots n]$):

create an array $tails[0 \sim n-1]$
create an array $idx[0 \sim n-1]$
create an array $prev[1 \sim n] = 0$
 $len = 0$

→ 維護目前長度為 $k+1$ 的遞增子序列中，最小尾元素值

→ $tail[k]$ 元素在原陣列 a 的索引。

→ $a[i]$ 在 LIS 中前一個元素的索引

算 LIS 長度和
prev

Time Complexity:
 $n \times O(\lg n)$
 $= O(n \lg n)$

for $i \leftarrow 1 \sim n$:

$k = \text{binary-search}(tails, len, a[i])$

→ 二分搜 $tails$ 中第一個 $> x$ 的位置，若都 $\leq$ 回傳 len
Time Complexity: $O(\lg n)$

$tails[k] = a[i]$

$idx[k] = i$

if $k > 0$:

$prev[i] \leftarrow idx[k-1]$

else:

$prev[i] \leftarrow 0$

if $k == len$

$len \leftarrow len + 1$

重建 LIS

Time Complexity:

$len \times O(1) = O(len)$

$O(len) \leq O(n)$

create an empty list result

$now \leftarrow idx[len-1]$

while $now \neq 0$:

result.push_back($a[now]$)

$now \leftarrow prev[now]$

return (len, result)

→ Total Algorithm Time Complexity: $O(n) + O(n \lg n) = O(n \lg n)$

<key> 維護一 array $tails$ ，使得 $tails[k]$ 為目前所見的元素中，長度為 $k+1$ 的 LIS 中最小的尾元素值。

發現到， $tails$ 永遠嚴格遞增，可以利用二分搜將每個元素處理時間降至 $O(\lg n)$ 。

Base Case $i=0$: $len=0$ ，loop invariant 皆成立。

Inductive Step:

$k = \text{binary-search}(tails, len, a[i])$ ，即 $tails$ 中第一個 $> a_i$ 的位置
分 2 Case 處理：

Case 1 $k=len$ ，所有 $tails$ 中元素皆 $\leq a_i$

$tails[len-1]$ 是某個長度為 len 的遞增子序列的尾巴且 $\leq a_i$ ，故將 a_i 接在其後得到長度為 $len+1$ 的遞增子序列。

a_i 是此長度中第一次出現的尾巴，改為最小。

更新後 len 變 $len+1$ ，loop invariant 依然成立。

Case 2 $k < len$ ， $tails[k] > a_i$ 且當 $k > 0$ 時 $tails[k-1] \leq a_i$

由歸納假設，存在長度為 k 、結尾為 $tails[k-1]$ 且 $\leq a_i$ 的遞增子序列，將 a_i 接在其後，得到長度為 $k+1$ 、結尾為 a_i 的遞增子序列。因 $a_i < \text{原本的 } tails[k]$ ，替換後仍必為最小尾巴。

$\therefore tails[k-1] \leq a_i \leq tails[k+1]$ ，遞增性依然維持。

Case 1、Case 2 下，loop invariant 皆成立。

由 loop invariant (3) 知，迴圈結束後， len 即整個陣列 LIS 的長度。重建 LIS 時由 $prev$ 構造， $a[idx[len-1]]$ 是長度為 len 的 LIS 的尾端，由 $prev$ 倒推能夠依序得長度遞減之合法前綴。

$\therefore$ 重建之子序列確實是一長度為 len 的遞增子序列。

Loop Invariant:

每輪 for loop 跑完後，以下 3 項必成立：

1. $tails[0 \sim len-1]$ 嚴格遞增

2. 對每個 $0 \leq k < len$ ， $tails[k]$ 會等於 $a_1 \dots a_i$ 中所有長度為 $k+1$ 的遞增子序列中，最小的尾元素值。

3. len 等於 $a_1 \sim a_i$ 中 LIS 的長度。